Campusmonk

PYQ : TCS & IBM

31. Next Permutation

17. Letter Combinations of a Phone Number

Campusmonk

31. Next Permutation

A permutation of an array of integers is an arrangement of its members into a sequence or linear order.

For example, for arr = [1,2,3], the following are all the permutations of arr:
[1,2,3], [1,3,2], [2,1,3], [2,3,1], [3,1,2], [3,2,1].

The next permutation of an array of integers is the next lexicographically greater permutation of its integers.
More formally, if all the permutations of the array are sorted in lexicographical order, then the next permutation of that array is the permutation that follows it in the sorted order.

If such arrangement is not possible (i.e., the array is the last permutation), then rearrange it to the lowest possible order (i.e., sorted in ascending order).

You must rearrange the numbers in-place and use only constant extra memory.

Example 1

> **Input:** nums = [1,2,3]
> **Output:** [1,3,2]
> **Explanation:**
> The next permutation of [1,2,3] is [1,3,2].

```java
public void nextPermutation(int[] nums) {
    int n = nums.length;
    int ind = -1;
    for(int i = n - 2; i >= 0; i--) {
        if(nums[i] < nums[i + 1]) {
            ind = i;
            break;
        }
    }
    if(ind == -1) {
        reverse(nums, 0, n - 1);
        return;
    }
    for(int i = n - 1; i > ind; i--) {
        if(nums[i] > nums[ind]) {
            swap(nums, i, ind);
            break;
        }
    }
    reverse(nums, ind + 1, n - 1);
    return;
}
private void swap(int[] nums, int i, int j) {
    int temp = nums[i];
    nums[i] = nums[j];
    nums[j] = temp;
}
private void reverse(int[] nums, int start, int end) {
    while(start < end) {
        swap(nums, start, end);
        start++;
        end--;
    }
}
```

```cpp
// Function to get the next permutation of given array
void nextPermutation(vector<int>& nums) {
    int n = nums.size(); // Size of the given array
    int ind = -1;
    for(int i = n-2; i >= 0; i--) {
        if(nums[i] < nums[i+1]) {
            ind = i;
            break;
        }
    }
    if(ind == -1) {
        reverse(nums.begin(), nums.end());
        return;
    }
    for(int i = n-1; i > ind; i--) {
        if(nums[i] > nums[ind]) {
            swap(nums[i], nums[ind]); // Swap with nums[ind]
            break;
        }
    }
    reverse(nums.begin() + ind + 1, nums.end());
    return;
}
```

```python
# Function to get the next permutation of given array
def nextPermutation(self, nums):
    n = len(nums)  # Size of the given array

    # To store the index of the first smaller element from right
    ind = -1

    # Find the first index from the end where nums[i] < nums[i+1]
    for i in range(n-2, -1, -1):
        if nums[i] < nums[i+1]:
            ind = i
            break

    ''' If no such index exists, array is in descending order
        So, reverse it to get the smallest permutation '''
    if ind == -1:
        nums.reverse()
        return

    # Find the element just greater than nums[ind] from the end
    for i in range(n-1, ind, -1):
        if nums[i] > nums[ind]:
            nums[i], nums[ind] = nums[ind], nums[i]  # Swap with nums[ind]
            break

    # Reverse the right half to get the next smallest permutation
    nums[ind+1:] = reversed(nums[ind+1:])
    return
```

Campusmonk

17. Letter Combinations of a Phone Number

Given a string containing digits from `2-9` inclusive, return all possible letter combinations that the number could represent. Return the answer in **any order**.

A mapping of digits to letters (just like on the telephone buttons) is given below. Note that 1 does not map to any letters.



Example 1:

> **Input:** digits = "23"
> **Output:** ["ad","ae","af","bd","be","bf","cd","ce","cf"]

Example 2:

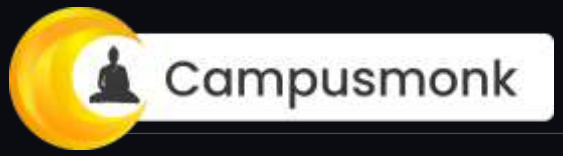
> **Input:** digits = "2"
> **Output:** ["a","b","c"]

Constraints:

- `1 <= digits.length <= 4`
- `digits[i]` is a digit in the range `['2', '9']`.

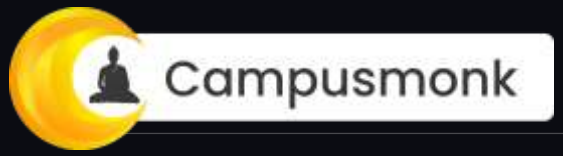
Java Auto

```java
class Solution {
public static void solve(String d , int inx , StringBuilder output , List<String>ans , String[]map){
if(inx==d.length()){
ans.add(output.toString());
return;
}
int num=d.charAt(inx)-'0';
String val=map[num];
for(int i=0;i<val.length();i++){
output.append(val.charAt(i));
solve(d,inx+1,output,ans,map);
output.deleteCharAt(output.length()-1);
}
}
    public List<String> letterCombinations(String digits) {
        List<String>ans=new ArrayList<>();
        if(digits.length()==0){
            return ans;
        }
        StringBuilder output = new StringBuilder();
        int index=0;
        String mapping[]={"","","abc","def","ghi","jkl","mno","pqrs","tuv","wxyz"};
        solve(digits,index,output,ans,mapping);
        return ans;
    }
}
```



main.cpp

```cpp
class Solution {
public:
    void solve(string digits, int index, string &output,
               vector<string> &ans, string mapping[]) {
        // Base case
        if (index == digits.length()) {
            ans.push_back(output);
            return;
        }
        int num = digits[index] - '0';
        string val = mapping[num];
        for (int i = 0; i < val.length(); i++) {
            // Choose
            output.push_back(val[i]);
            // Recursive call
            solve(digits, index + 1, output, ans, mapping);
            // Backtrack
            output.pop_back();
        }
    }
    vector<string> letterCombinations(string digits) {
        vector<string> ans;
        if (digits.length() == 0) {
            return ans;
        }
        string output = "";
        int index = 0;
        string mapping[] = {
            "", "", "abc", "def", "ghi",
            "jkl", "mno", "pqrs", "tuv", "wxyz"
        };
        solve(digits, index, output, ans, mapping);
        return ans;
    }
};
```



main.py

```python
class Solution:
    def solve(self, digits, index, output, ans, mapping):
        # Base case
        if index == len(digits):
            ans.append("".join(output))
            return
        num = int(digits[index])
        val = mapping[num]
        for ch in val:
            # Choose
            output.append(ch)
            # Recursive call
            self.solve(digits, index + 1, output, ans, mapping)
            # Backtrack
            output.pop()
    def letterCombinations(self, digits: str):

        ans = []

        if len(digits) == 0:
            return ans

        output = []
        index = 0

        mapping = [
            "", "", "abc", "def", "ghi",
            "jkl", "mno", "pqrs", "tuv", "wxyz"
        ]

        self.solve(digits, index, output, ans, mapping)

        return ans
```